Ingegneria del Software 21/07/2022 Soluzione del Tema

Esercizio 1

Si vuole progettare un'applicazione Java per gestire aste online. A tale scopo, si è specificata la classe `AuctionManager`, riportata più avanti (molto semplificata rispetto alla realtà, p.es. nella gestione delle scadenze o nel tracciamento dei partecipanti).

Ad ogni cliente è associato un customer id univoco e segreto (modellato per semplicità con un `long`), con cui i clienti possono sia vendere che fare offerte. La gestione dei clienti è gestita da classi opportune, che sono ignorate qui.

Per mettere in vendita un oggetto, si deve passare al metodo `addItem` un oggetto della classe immutabile `Item`, oltre al customer id del venditore. È responsabilità del venditore creare e descrivere l'oggetto corrispondente, tramite i metodi della classe `Item` (in gran parte ignorati qui):

```
public /*@ pure @*/ class Item {
    public Item(long sellerId, ...altri parametri ignorati)
    ...altri metodi ignorati
    //restituisce il customer id di chi ha creato l'oggetto
    public long sellerId();
    // Restituisce la base d'asta per l'item
    public double getStartingPrice();
}

public class AuctionManager {
    // Costruisce un gestore di aste, senza ancora alcun oggetto in vendita.
    public AuctionManager();

    // Il cliente con codice sellerId mette in vendita l'oggetto item, non ancora messo in asta,
    // Se l'item corrisponde ad un oggetto gia' in asta, rilancia WrongItemException;
    public void addItem(Item item, long sellerId) throws WrongItemException;

    // Il venditore con sellerId che ha messo in vendita l'oggetto item, accetta
    // la migliore offerta corrente. Il metodo ritorna la migliore offerta corrente
    // e ritira (rimuove) l'oggetto dall'asta.
    // Se non vi sono state offerte, lancia eccezione NoBidException.
    // WrongItemException e' lanciata se il sellerId non corrisponde all'item
    // o se l'item non risulta in asta.
    public double knockDown(Item item, long sellerId) throws WrongItemException, NoBidException;

    // Il cliente con buyerId propone un'offerta di valore newBid, superiore alla massima
    // corrente (altrimenti si lancia BidTooLowException), per l'oggetto item in asta.
    // Se l'item non e' in asta si lancia  WrongItemException
    public void bid(Item item, long buyerId, double newBid) throws WrongItemException,
           BidTooLowException;

    // Restituisce l'insieme di tutti gli oggetti attualmente in asta
    public /*@ pure @*/ Set<Item> auctionedItems();

    // Restituisce la migliore offerta corrente per l'item (se e'  in asta).
    // Se non vi sono state offerte restituisce la base d'asta.
    // Se l'item non e' in asta si lancia  WrongItemException
    public  /*@ pure @*/ double getBestBid(Item item) throws WrongItemException;
}
```

Domanda a)

Specificare in JML il metodo `bid`. Si consiglia di introdurre abbreviazioni per rendere la specifica più semplice e leggibile.

Soluzione

`otherItemsUnchanged(item)` sta per

```
(\forall Item it; auctionedItems().contains(it) && it!=item;
   getBestBid(it)==\old(getBestBid(it)) &&
   \old(auctionedItems()).contains(it)) &&
(\forall Item it; \old(auctionedItems()).contains(it) && it!=item
   auctionedItems().contains(it))
```

`unchanged(item)` sta per

```
getBestBid(item)==\old(getBestBid(item)) &&
auctionedItems().contains(item) && \old(auctionedItems()).contains(item)
```

```
/*@ ensures \old(auctionedItems()).contains(item) && auctionedItems().contains(item) &&
    newBid>\old(getBestBid(item)) && newBid==getBestBid(item) &&
    otherItemsUnchanged(item)

    signals(WrongItemException e) !\old(auctionedItems().contains(item)) &&
    !auctionedItems().contains(item) && otherItemsUnchanged(item)

    signals(BidTooLowException e) newBid<=getBestBid(item) &&
    unchanged(item) && otherItemsUnchanged(item)
@*/
public void bid(Item item, long buyerId, double newBid) throws WrongItemException, BidTooLowException;
```

Domanda b)

Specificare il metodo `knockDown`.

Soluzione

```
/*@ ensures item.sellerId()==sellerId && \old(auctionedItems().contains(item)) &&
    !auctionedItems().contains(item) && \result==\old(getBestBid(item)) &&
    \old(getBestBid(item))>getStartingPrice(item) && otherItemsUnchanged(item)

    signals(WrongItemException e) (otherItemsUnchanged(item) &&
    !(\old(auctionedItems()).contains(item) && !auctionedItems().contains(item)) ||
    (item.sellerId()!=sellerId && unchanged(item))

    signals(NoBidException e) \old(getBestBid(item))==getStartingPrice(item) &&
    otherItemsUnchanged(item) && unchanged(item);
@*/
public knockDown(Item item, long sellerId) throws WrongItemException, NoBidException;
```

Domanda c)

Si consideri l'implementazione seguente del rep:

```
private Set<Item> currentItems;     // gli item all'asta
private Map<Item, Double> bestBids; // l'offerta migliore per ciascun item;
```

Si ricorda che una Map<K, V> è una struttura dati che associa a chiavi di tipo K (Item in questo caso) valori di tipo V (Double in questo caso). Una Map fornisce, fra gli altri, i metodi osservatori get(K key), che restituisce il valore univoco associato alla chiave key, e containsKey(K key), che restituisce true se esiste un valore associato alla chiave key. Altri metodi sono ad esempio size() e isEmpty(), di significato ovvio.

Si scrivano l'invariante di rappresentazione e la funzione di astrazione relative a questo rep.

Soluzione

RI:

```
private invariant
currentItems!=null && bestBids!=null &&
!bestBids.containsKey(null) &&
(\forall Item it; currentItems.contains(it); bestBids.containsKey(it) &&
(\forall Item it; bestBids.containsKey(it)); currentItem.contains(it))
```

AF:

```
private invariant
auctionedItems().size()==currentItems.size() && auctionedItems().containsAll(currentItems) &&
(\forall Item it; currentItems.contains(it); getBestBid(it)==bestBids.get(it))
```

Domanda d)

Si vuole aggiungere alla classe AuctionManager un metodo per consentire al venditore, qualora non vi siano state offerte, di introdurre uno sconto sulla base d'asta:

```
public void discount(Item item, long sellerId, double discount)
            throws WrongItemException, AlreadyBiddedException;
```

È possibile definire, senza violare il principio di sostituzione, una sottoclasse di AuctionManager che abbia anche questo metodo? Motivare la risposta.

Soluzione

Non è possibile in quanto il nuovo metodo viola un'importante proprietà evolutiva della classe: invocazioni successive del metodo getBestBid() per uno stesso `item` ritornano valori monotoni non decrescenti. L'introduzione del metodo discount() permetterebbe invece di ridurre il valore ritornato da due invocazioni di getBestBid(), eseguite rispettivamente prima e dopo l'invocazione di discount(). NB: Lo sconto non modifica la base d'asta memorizzata nell'item (in quanto immutabile), ma il valore ritornato da getBestBid() quando non vi sono ancora state offerte: getBestBid(item) in questo caso dovrebbe ritornare item.getStartingPrice()*discount(item) invece di item.getStartingPrice().

Esercizio 2

Si consideri la seguente classe `ScoreBoard` per memorizzare i voti di un insieme di studenti passato in fase di costruzione:

```
public class ScoreBoard {
  private String[] students;
  private int[] scores;
  public ScoreBoard(String[] stud) {
    students = new String[stud.length];
    for(int i=0; i<stud.length; i++) students[i] = stud[i];
    scores = new int[stud.length];
  }
  public int getScore(String stud) {
    for(int i=0; i<students.length; i++) {
      if(students[i].equals(stud)) return scores[i];
    }
    return -1;
  }
  public void setScore(String stud, int score) {
    for(int i=0; i<students.length; i++) {
      if(students[i].equals(stud)) scores[i] = score;
    }
  }
}
```

Domanda a)

Si introduca l'opportuno codice di sincronizzazione nei metodi `getScore` e `setScore` al fine di massimizzare il parallelismo, consentendo a thread diversi di accedere in parallelo ai dati di studenti diversi, evitando conflitti nell'accesso ai dati (voto) del medesimo studente.

Soluzione

L'array `students` è immutabile. Possiamo scorrerlo senza rischio di conflitti in accesso e possiamo usarlo come array di oggetti su cui sincronizzarsi quando si deve leggere o scrivere un voto.

```
public class ScoreBoard {
  private String[] students;
  private int[] scores;
  public ScoreBoard(String[] stud) {
    students = new String[stud.length];
    for(int i=0; i<stud.length; i++) students[i] = stud[i];
    scores = new int[stud.length];
  }
  public int getScore(String stud) {
    for(int i=0; i<students.length; i++) {
      if(students[i].equals(stud)) synchronized(students[i]) { return scores[i]; }
    }
    return -1;
  }
  public void setScore(String stud, int score) {
    for(int i=0; i<students.length; i++) {
      if(students[i].equals(stud)) synchronized(students[i]) { scores[i] = score; }
    }
  }
}
```

NB: la sincronizzazione sulla String stud, invece, puo' portare ad errori di sincronizzazione, in quanto Java non garantisce che due stringhe uguali siano sempre lo stesso oggetto! Java usa automaticamente lo "string interning" per espressioni del tipo:

String s1=="John", String s2=="John" (quindi s1==s2),

ma non per String s3=new("John") (quindi s1!=s3).

Domanda b)

Si modifichi il metodo `getScore` affinchè sospenda il chiamante se il voto dello studente cercato è minore o uguale a zero. Si specifichi se e come sia necessario modificare altri metodi.

Soluzione

```
public class ScoreBoardSync1 {
  private String[] students;
  private int[] scores;
  public ScoreBoardSync1(String[] stud) {
    students = new String[stud.length];
    for(int i=0; i<stud.length; i++) students[i] = stud[i];
    scores = new int[stud.length];
  }
  public int getScore(String stud) throws InterruptedException {
    for(int i=0; i<students.length; i++) {
      if(students[i].equals(stud))
        synchronized(students[i]) {
          while(scores[i]<=0) students[i].wait();
          return scores[i];
        }
    }
    return -1;
  }
  public void setScore(String stud, int score) {
    for(int i=0; i<students.length; i++) {
      if(students[i].equals(stud))
        synchronized(students[i]) {
          scores[i] = score;
          students[i].notifyAll();
        }
    }
  }
}
```

Esercizio 3

```
public static int f(int n) {
  if (n <= 0 || n > 6) return 0;
  if (n\%2 != 0) n = n + 1;
  int a = 0;
  while (n > 1) {
    a = a + n;
    n = n - 2;
  }
  return a;
}
```

Si determini per la funzione `f()` un insieme minimo di casi di test per ciascuno dei seguenti criteri di copertura.

(a) Copertura delle istruzioni (statement coverage)

(b) Copertura delle decisioni (edge coverage)

(c) Copertura dei cammini (path coverage)

Soluzione

(a) Sono necessari (e sufficienti) due casi, uno che entri nel primo `if` e uno che copra tutte le successive istruzioni, sia quelle all'interno del corpo del secondo `if`, sia quelle all'interno del `while`. Il primo caso deve avere `n <= 0` oppure `n > 6`, ad esempio `n = 0`. Il secondo caso deve avere `n` dispari, ad esempio `n = 1`.

(b) Ai casi precedenti è necessario aggiungere un caso con `n` pari, per valutare negativamente la condizione del secondo `if`, ad esempio `n = 2`.

(c) In tutto sono possibili 7 cammini:

(1) Un primo cammino entra solo nel primo `if`, ad esempio con `n = 0`.

(2) Con `n = 1` si entra nel secondo `if` e si entra nel ciclo `while` 1 volta.

(3) Con `n = 2` NON si entra nel secondo `if` e si entra nel ciclo `while` 1 volta.

(4) Con `n = 3` si entra nel secondo `if` e si entra nel ciclo `while` 2 volte.

(5) Con `n = 4` NON si entra nel secondo `if` e si entra nel ciclo `while` 2 volte.

(6) Con `n = 5` si entra nel secondo `if` e si entra nel ciclo `while` 3 volte.

(7) Con `n = 6` NON si entra nel secondo `if` e si entra nel ciclo `while` 3 volte.

Esercizio 4

Si considerio le seguenti classi Java, in cui viene omessa l'ovvia implementazione dei metodi osservatori `getMembers()`, `getAge()`, `getName()`.

```
public class Group {
  public List<Person> getMembers() { ... }
}

class Person {
  public String getName() { ... }
  public int getAge() { ... }
}
```

Si risponda ai seguenti quesiti facendo uso *esclusivamente* di costrutti della programmazione funzionale.

Domanda a)

Si completi l'implementazione del seguente metodo per fare in modo che stampi l'età media delle persone appartenenti al gruppo. Se il gruppo è vuoto, il metodo non deve stampare nulla.

```
public static void printAverage(Group group) {
  ...
}
```

Soluzione

```
public static void printAverage(Group group) {
  group.getMembers().stream()
  .mapToInt(p -> p.getAge())
  .average()
  .ifPresent(System.out::println);
}
```

Domanda b)

Si completi l'implementazione del seguente metodo per fare in modo che ritorni una lista contenente i nomi di tutte le persone maggiorenni presenti all'interno dei gruppi in `groups`.

```
public static List<String> namesInGroups(List<Group> groups) {
  return ...
}
```

Soluzione

```
public static List<String> namesInGroups(List<Group> groups) {
  return groups.stream()
  .flatMap(g -> g.getMembers().stream())
  .filter(p -> p.getAge() > 18)
  .map(p -> p.getName())
  .collect(Collectors.toList());
}
```